[ADR] Authentication and authorization

Status

Proposed by @Ulad Palinski

Stakeholders:

Outcome

TBD

Context

Access type

Authorized access

In most scenarios, it's assumed that clients requests (read/write) to the storage and CDN nodes should be authorized. Requests must be sent by bucket owner or the person to whom the owner has granted access.

The access can be granted not only for read operations but also for write (delete). One of the most basic use case that we can think about is a decentralized iCloud where bucket owner can share an access to the bucket with his family so any member can upload/download data.

Public access

In theory, there can be use cases when content owner would like to distribute a content for 'free' and pay for the content delivery by his own (e.g. decentralized YouTube) but this scenario adds additional complexity related to protection against (1) malicious clients that abuse this free access and (2) malicious nodes that can fake activity reports and try to earn more funds. Although the described 'public access' is out of the scope of this proposal, it would be useful to think

about possible support of those scenarios in future and its consequences during proposal preparation.

Access control

First-party authorization

The most basic and straightforward approach to control an access is to grant an access directly to a storage. So Alice gives an access to a bucket or some content stored in the bucket to a Bob.

Third-party authorization

More advanced scenarios would benefit from the ability to share an access based on third party attestation.

One of the scenarios here can be a content referenced by NFT. So once user buys an NFT he automatically receives an access to the content referenced by that NFT that is stored in our storage.

Another use case can be a Netflix like application where user receives an access to the content after buying a subscription. The fact of having a subscription that is stored in other system could be used to authorise user access in the DDC.

In the Cere such 3rd party can be an NFT Global Registry.

Ideally DDC shouldn’t have any 3rd party specific logic (e.g. DDC shouldn’t directly depend on NFT Global Registry).

Might be ZK proofs can somehow help to support such mechanism.

Access without authorization

In case if bucket is public and user, who has token, is requesting reading the stored content, we can skip token verification, if bucket is paid (owner of bucket has no debts of this bucket)

Access granularity

Bucket access

The bucket access is the highest level of granularity. This access allows clients to access any content stored in the bucket.

Piece access

The piece is a unit of data stored in DDC. So it is the lowest level of granularity. This access allows clients to access only this particular piece (content) in the bucket. It can be an image, video, document etc.

Collection access

The bucket and piece accesses are not very flexible and allows give an access to either whole content (what can be too much) or single piece (what can add extra control cost).

It would be convenient for applications to have an ability to grant an access to a collection of pieces. Even more convenient would be to support granting an access to a collection of pieces that can be represented as hierarchy.

The use cases here can be (1) again a Netflix like application that can grant an access to series that have seasons and episodes inside (2) Google Drive like application where users can share an access to folders (that have hierarchy inside).

DAG API provides an ability to link data and represent it as a graph. The use cases described above can be supported in case there is a way to determine the parent node to which client have an access. The possible way is to access node by path (e.g. `1edf…24g/season_1/episode_1.mp4`) so entry point is a piece `1edf…24g` to which client have an access so it means that he also have an access to both `season_1` and `episode_1.mp4`. The issue here is unclarity on how to authenticate hierarchical content because client requests a content by a path and he doesn't know the CID of `episode_1.mp4` so it can be tampered.

Considerations

CDN to Storage

Access to the content that is distributed via CDN also should be authenticated and authorized. In case requested content isn't yet cached on CDN layer, requests should be passed to the storage layer to retrieve content and cache it in case its profitable for CDN nodes.

Due to the simplicity and the need to proxy requests, it would be nice to have the same or similar authentication/authorization mechanism for both CDN and storage. Might be client credentials should be extended by CDN node credentials (e.g. to distinguish between direct client requests and proxied by CDN).

Activity capture (DAC)

Client requests suppose to contain some metadata and signatures to authorise request. The signature of authorised request proves that client has sent a request and he has a right to access content and there is no way to fake such request otherwise the authorising party would reject it.

In theory that proof (request signature-based authorisation) can be re-used for activity capture needs. So, client adds authorisation details to the request. The CDN or storage uses that details to authorise the request. Also the node could store authorisation details for activity capture needs and provide it to some auditor or validator that can verify that the client actually send such a request and there is no way to fake the requests.

Summary

The considerations outlined above allow us to determine a functional and non-functional requirements.

Functional requirements

- Only authorized requests can be processed by the CDN and storage nodes.
- The client can provide access to his content to another client.
- Read access can be granted to a whole bucket, single piece or collection of pieces.
- Write/delete access can be granted to a whole bucket only.
- If bucket is public and has no debts, no verification is needed
- @Ulad Palinski can the access be to any user without a wallet - like youtube?
- [TO BE DISCUSSED] Client can revoke an access to a bucket or some content inside.

Non-functional requirements (quality attributes)

- **Scalability**. While write and delete access isn’t expected to be granted to large number of clients, the read access in theory can be granted to millions or even billions of clients. So access granting (control) mechanism should be scalable.

- **Effectiveness**. In addition to scalability, the solution should be efficient in terms of cost and delay. Ideally access should be granted immediately and at no cost (e.g. blockchain-based solution have a transaction costs).
- [TO BE DISCUSSED] **Revocability**. Bucket owner can revoke an access (e.g. client overused the content) or access owner can revoke it (access leaked).
- **Flexibility**. Access can be granted on different granularity level and operation types (e.g. read particular collection of pieces or delete particular piece).
- **Security**. Credentials sent to the protocol should be short living. The temporary credentials should have a client signature, so leaked credentials shouldn't be valid without client's signature. Attacker should have client's private key to have a long term access.
- **Complexity**. Ideally DDC should provide scalable, flexible and efficient solution to manage an access control with minimal client effort.
- **Polkadot.js compatibility**. In web application client suppose to interact with DDC via `polkadot` extension. The auth mechanism should take it into account. `polkadot` shows the messages to sign to the client. So, ideally they should be human readable but might be it's not a strict requirement. Also `polkadot` wraps the message into `<Bytes></Bytes>` tag to be protected against some kind of attacks (e.g. some applications can ask to sign token transfer transaction but this `Bytes` tag would make it invalid).

Decision

Options

Pallet (smart contract) based access control

The pallet is a starting point because bucket owner defined during bucket creation. So the bucket owner has the root access that is stored in the pallet.

In theory, pallet that holds bucket information also can manage an access control. So bucket owner could grant an access in the pallet and bucket would have a list of clients and their access (e.g. read/write etc.)

Despite the fact it can work well for a use cases like iCloud or some company that grants an access to the buckets to its employees only, unfortunately it doesn't

support use cases where access should be granted to a huge amount of clients like video streaming/sharing platform or game application because of (1) cost overhead and (2) scalability issues due to the need to perform a transaction on chain.

Intermediate proxy service

As an option, client (most probably application developer) can set up his own service that can authenticate and authorise client in his own identity service and proxy get/put requests to the DDC. In this case DDC always receives requests from application developer account and access control on DDC layer can be managed via pallet. The access control and granting to final clients is a responsibility of application developer and DDC doesn't know anything about it. In the pallet an access granted to the application developer only.

The issue in this approach is that client (application developer) have to take care about (1) identity and (2) intermediate proxy services. The difficulty in the identity service is that it should have non-trivial logic related to access control and difficulty in the intermediate proxy service is that it should be able to process a lot of traffic.

Token based access delegation

The `decentralised delegation` between principles assumes that client that have an access to a bucket (or particular content inside it) can delegate (grant) access to someone else.

Credential (granted access) can be something similar to JWT that contains details about the issuer, claims and signature. The client to whom the access was granted (received JWT-like token) should wrap that token and sign it in order to prove the right to use that token. To make it more flexible, we can add the ability to add own claims that are more limited that was granted to the client.

In a result, such recursive JWT-like token can be treated as a trust chain that proves that client presented the final token (who sent a request to the DDC) has been granted by the client that have an access to the bucket on the pallet layer. Despite the fact that token could be generated by the set of delegatee (to whom the access was delegated) the whole chain is traversable and provable up to the public key that have a root or admin access (stored on chain).

Proposal

Description

To get the best of the options described above, the DDC can adopt combined solution: `pallet based access control` + `token based access delegation` + `identity service`.

The idea is to have high-level access control on the pallets level. So after the bucket creation, the wallet that created bucket becomes a bucket owner that have the root access. After creation bucket owner can grant an access to some other client in pallet (on-chain transaction). Such a pallet level access have next pros (revocability) and cons (transaction cost and slowness).

To address scalability, effectiveness and flexibility issues `token based access delegation` can be added. The idea is to allow clients that have a pallet level access to delegate an access by issuing signed access token to other clients (subject). So other client can provide original token signed by client that have an access to the bucket on pallet level + own signature to proof the right to use that access token. It can be represented as a recursive access token where one signed token wraps another one.

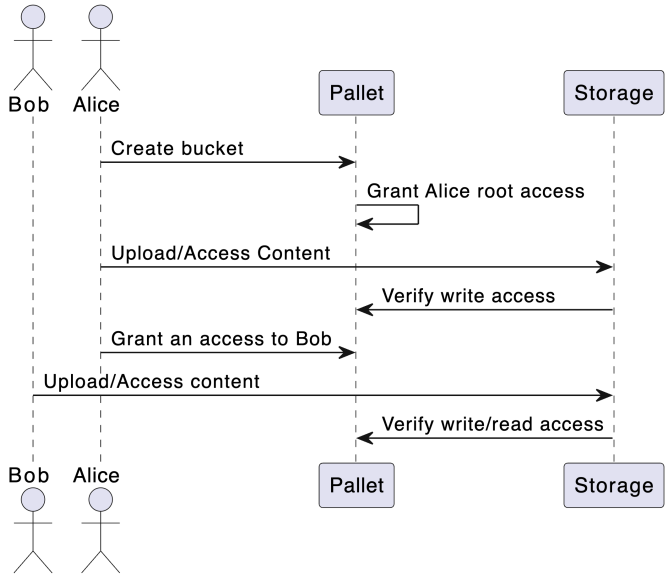
Token based access delegation requires client to generate such tokens what isn't very convenient in some real-world scenarios. One of the use cases is a Cere's Freeport project that would like to grant an access to a content when some client bought an NFT. So the fact of NFT ownership should give an access to the new owner to access the data stored in DDC. Also ideally the auth mechanism should provide an ability to grant an access automatically without client involved in the process (because client can be offline and in general he doesn't want to generate a token on each NFT sale). The idea here is that application developers (e.g. Freeport app devs) can implement their own identity services that can be granted access by bucket owners (e.g. content creators) to delegate an access to other clients based on application specific logic (e.g. NFT global registry in Freeport use case).

Summary

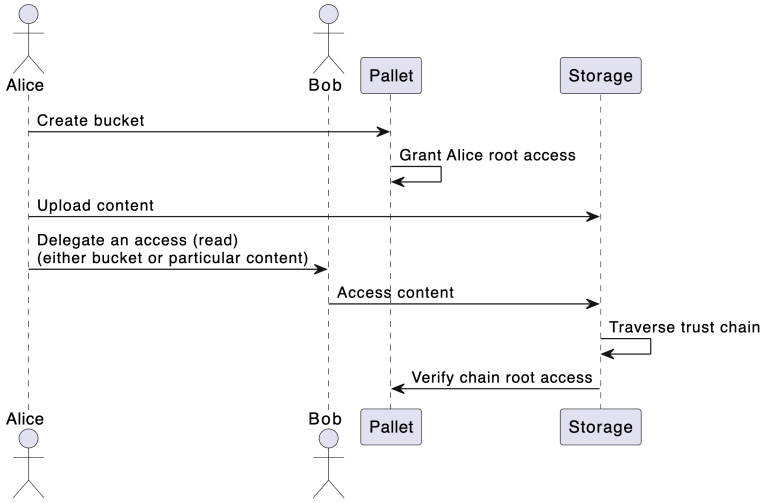
Recursive JWT-like signed tokens that represent a `trust chain` where first token is issued by the client that have an access on pallet level and last token is signed by the client that sends a request to DDC.

Sequence diagrams

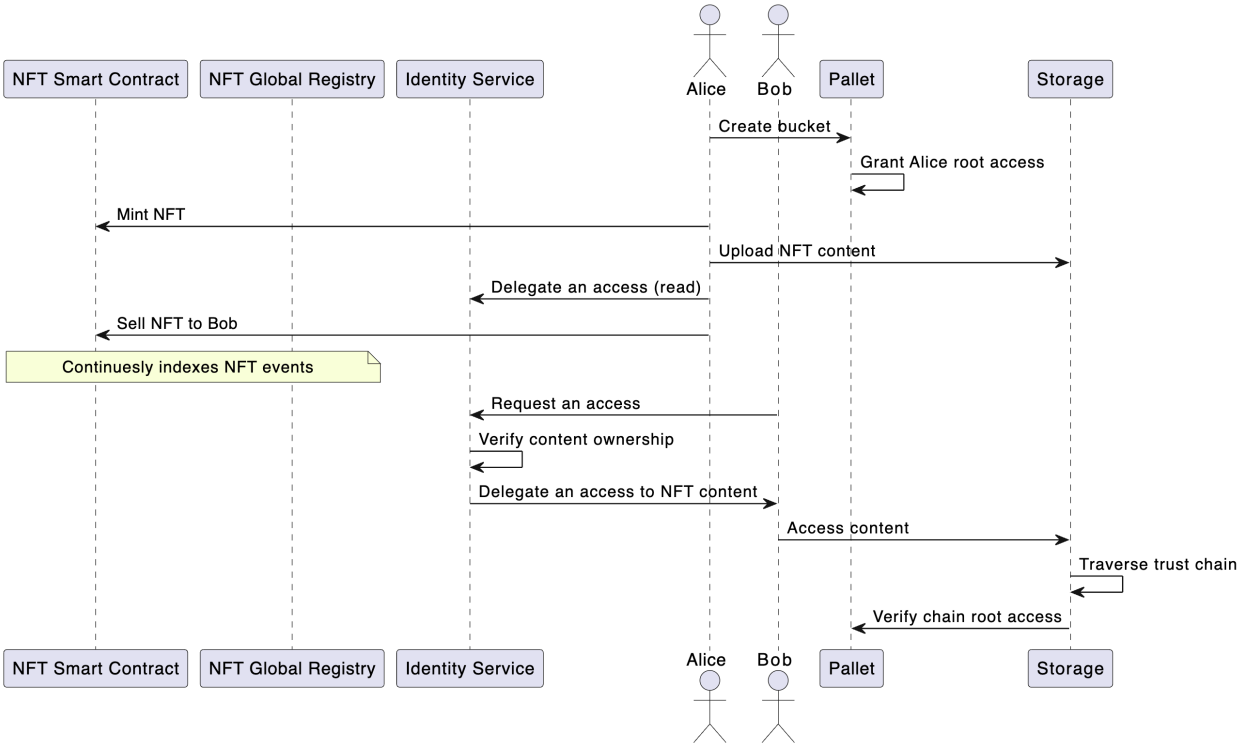
1. Pallet based access control



2. Token based access delegation



3. Token based access delegation + identity service (Freeport use case)



Token specification (.proto)

```
message Signature {
  Algorithm algorithm = 1;
  bytes signer = 2;
  bytes value = 3;

  enum Algorithm {
    ED_25519 = 0;
    SR_25519 = 1;
  }
}

// Token represents trust chain of tokens where first token i
s verifiable by bucket access on chain and last token is gene
rated by the client that sends a request
message AuthToken {
  Signature signature = 1; // signature signer is an issuer.
issuer of first token should have an access on pallet level a
```

```
nd subsequent tokens can skip 'issuer' and take 'subject' fro
m previous token to verify signature
  Payload payload = 2;
}

message Payload {
  optional AuthToken prev = 1; // prev token in trust chain
(based on known use cases max depth can be limited to 3 or in
crease to 5 to support more potential use cases)
  optional bytes subject = 2; // whom. every except last toke
n should be non empty. next token should be signed by this su
bject
  optional bool canDelegate = 3; // subject can be prohibited
to delegate access to anyone else (next token should be last)

  optional uint64 bucketId = 4; // mentioned only once in tru
st chain (or even not mentioned at all if bucket owner decide
d to share access to all his buckets)
  repeated Operation operations = 5; // each next token in tr
ust chain should have less or equal privileges (e.g. token re
stricted to 'get' operation can't have 'put' in next token)
  optional int64 expiresAt = 6; // each next token should exp
ires earlier or at the same time as previous one (e.g. token
can't have lower expiresAt than in next token)
  optional bytes pieceCid = 7; // mentioned only once in trus
t chain (in DAG API nested pieces can be accessed by path)
}

enum Operation {
  UNKNOWN = 0;
  PUT = 1;
  GET = 2;
  DELETE = 3;
}
```

SDK [pseudocode]

```
const main = async () => {
    // Init Alice client
    const ddcClientAlice = await DdcClient.buildAndConnect({c
lusterId: 1, palletAddress: TESTNET}, MNEMONIC_ALICE);
    // Alice creates bucket and receives root access
    const {bucketId} = await ddcClientAlice.createBucket(clus
terId);
    // Alice can upload/download content (only root token gen
erated inside)
    const nftCid = await ddcClientAlice.upload(bucketId, "Hel
lo world NFT!");

    // Alice delegates a token based access to identity servi
ce
    const idSvcToken = new Token(bucketId, "read", expiresIn1
Year, PK_ID_SVC).sign(MNEMONIC_ALICE)
    // Alice sends token to identity service
    await idSvcClient.sendToken(idSvcToken)
    // Or Alice can delegate pallet based access to identity
service (in such case it will be revocable)
    await ddcClientAlice.grantAnAccess(bucketId, "read", PK_I
D_SVC)

    // Bob requests access in identity service that delegates
access after verifying that Bob is a new NFT owner
    const bobToken = idSvcClient.getToken(nftCid) // new Toke
n(expiresIn1Day, nftCid, PK_BOB).prev(idSvcToken).sign(MNEMON
IC_ID_SVC))

    // Init Bob client
    const ddcClientBob = await DdcClient.buildAndConnect({clu
sterId: 1, palletAddress: TESTNET}, MNEMONIC_BOB);
    // Bob can download content (bobToken is extended by new
token signed by MNEMONIC_BOB)
    const nftContent = await ddcClientBob.download(bobToken,
```

```
bucketId, nftCid)
}
```

Token example (JSON serialised to make it more presentable)

```
{
  "signature": {
    "algorithm": 0, // ED_25519
    "signer": "BOB_PUBLIC_KEY",
    "value": "SOME_VALUE_BOB_SIGNATURE"
  },
  "payload": {
    "prev": {
      "signature": {
        "algorithm": 0,
        "signer": "ALICE_PUBLIC_KEY",
        "value": "SOME_VALUE_ALICE_SIGNATURE"
      },
      "payload": {
        "subject": "BOB_PUBLIC_KEY",
        "canDelegate": true,
        "bucketId": 123,
        "operations": [1, 2],   // PUT, GET
        "expiresAt": 1672531199,
        "pieceCid": "some_piece_id"
      }
    },
    "operations": [2],  // GET
    "expiresAt": 1672535999,
  }
}
```

Content sharing (public access)

In theory, such authentication mechanism support use case when client wants to share his content via sending link (e.g. share an image) that is already signed by

the client (analogy is `pre-signed url` in S3 API based storages). The issue here is that except from token, DDC protocol expects `signed req/ack` to capture activity and reward the network.
In general, such use case can be adopted via introducing some layer of trust. E.g. Cere's nodes can be deployed under some dns name (e.g. `ddc.cere.network`) and on client's decision he can put in shared url not only token but also signed `req` that doesn't have a `request_id` (request can be resent) and sets `no_ack` field to true (so DAC doesn't expect `ack` and reward based on `req` only). This introduces security vulnerability because such requests can be used to produce `replay attack` but it can be clients decision to trust our nodes. Also shared link would have some expiration time so it can't be 'replayed' forever.

Notes

- Only pallet based access is revocable.
- Infinite (or almost infinite) tokens can be a problem. Protocol should support them until expiration. Might be token's TTL can be limited by protocol (e.g. 1 month). So, permanent or long-lived access can be granted on SC level and in case protocol authentication mechanism changes, migration period would be 1 month (until every generated token expires).
- Granular access (per piece or collection of pieces via DAG) can be granted only via token. Otherwise solution isn't efficient/scalable/etc. because need to keep access control record per each piece on chain.
- Those tokens should be somewhere stored. Client can have an access to many different pieces and have a lot of tokens. In Freeport use case an entry point can be identity service. In theory other use cases could store tokens in DDC or even blockchain (remember that tokens doesn't have value until they are 'wrapped/signed' by the subject's private key).
- Polkadots.js compatibility is out of scope. Assumption is that any text can be shown in browser extension window and apps can do some workaround to show human readable representation of what exactly user is going to sign. The simplest approach to support it later would to add `Source` enum in token's `Header` (next to `algorithm` and `issuer`) where default value is smth like `NONE` and other is `POLKADOT_JS`. In case it's set to `POLKADOT_JS` the signature verifier wraps serialised payload by `<Bytes></Bytes>` tag.

- Most probably CDN → Storage authentication will be part of the activity capture flow. Tokens proposed in this document are going to proof that client has an access to the content. CDN and Storage APIs are going to be the same at the moment.

Consequences

- Trust chain where root token is secured by access on pallet (blockchain) level.
- Each link in trust chain has a signature that secures whole trust chain and it can't be broken. Similar as blockchain but in our case tokenchain 🙂
- Trust chain roots are revocable on pallet level. It means that access can be revoked to all chains issued by that root at one operation.
- Efficient and scalable tokens delegation allows to support use cases like video streaming platform (or e.g. social media) when content can be accessed by millions of users (application developer should develop his own identity service to issue tokens).